# Test datasets – Code Plagiarism Checker

Small, labelled sets of Java files for the test cases T-01 to T-13 (Group 8, SDN260S). Each folder is one test case. Give the files in a folder to the program, run the comparison, and compare what you see with the expected result below. Record the outcome in `Group8_Test_Log_and_Evaluation.xlsx`.

These files are **inputs for the checker**, not one project. Several folders contain a file called `Student_A.java`, so do not compile a whole folder together. Every `.java` file compiles on its own.

## The nine datasets

| Folder | Test | Files | Expected | Result on the final build (1 Nov 2026) |
|---|---|---|---|---|
| T01_different_programs | T-01 Two different programs | Student_A.java (average of temperatures), Student_B.java (palindrome checker) | Low similarity, not flagged | 20.0%, not flagged |
| T02_similar_programs | T-02 Two similar programs | Student_A.java (bubble sort), Student_B.java (same sort, variables renamed, comment changed, one extra print line) | High similarity, flagged | 91.3%, flagged |
| T03_multiple_submissions | T-03 Multiple submissions | Student_A.java to Student_E.java. A and B are the similar pair; C, D and E are unrelated | All 10 pairs compared, sorted high to low, A and B first | 10 pairs, A and B first at 91.3% (only flag); the rest 15.6% to 29.0% |
| T04_empty_file | T-04 Empty file | Student_A.java (valid), Empty.java (0 bytes), Blank_Lines.java (only whitespace) | Clear message, no crash, empty files skipped | Both rejected with "File is empty", no crash |
| T05_invalid_file | T-05 Invalid file | Student_A.java, Student_B.java (valid); notes.txt, results.csv, Student_C.jav (not .java) | The three invalid files rejected with a readable error; the valid pair still compares | Rejected with "Only .java files can be checked"; valid pair 20.0% |
| T06_duplicate_file | T-06 Duplicate file | Student_A.java, Student_B.java (identical) | 100%, flagged | 100.0%, flagged |
| T07_very_small_file | T-07 Very small file | Tiny_A.java, Tiny_B.java (different one-line classes), Only_Comments.java (no code once comments are removed) | No divide-by-zero, no false 100% | 8.3%, 0.0%, 0.0%; no crash |
| T08_large_file | T-08 Large file | Large_A.java, Large_B.java (about 1,300 lines each; 10 of 100 methods rewritten in B) | Completes, time recorded | 86.2%, flagged; about 12 ms for the comparison itself |
| T09_different_formatting | T-09 Different formatting | Student_A.java, Student_B.java (same code, different spacing, braces and comments) | Same code with other layout scores high | 100.0%, flagged |

T-10 to T-13 (processing time, error messages, CLI usability, GUI usability) reuse these files. The official wording of those four tests has not been seen, so they follow the split described in the `Test Cases` sheet.

## How to run a dataset

From the project folder, after compiling (see the project README):

```
java -cp target/classes plagiarismchecker.Main test_datasets/T03_multiple_submissions
```

A folder is scanned, sub-folders included. To give files one at a time, list them after the class name. On Windows use `target\classes` and backslashes in paths. Alternatively start the text menu with `--cli` or the window with `--gui` and add the files there.

## Notes for testers

- **T-04 and T-05:** the rejection message appears on screen when you give the files on the command line. In the text menu and in the window it is written to `logs/plagiarism-checker.log` instead (DEF-10 in the log). When a whole folder is scanned, the non-`.java` files in T-05 are skipped without a message (DEF-12).
- **T-07:** `Tiny_A.java` and `Tiny_B.java` are deliberately different, so 100% for them would be a false match. `Only_Comments.java` has no code left after cleaning, so it scores 0.0% against everything, silently (DEF-13).
- **T-08:** if the pair is too easy for your computer, make a bigger one by adding methods, and record the line count with the time.
- **T-06:** the two files have the same SHA-256 (`37a48ee3cebb4743e405eb7306945865dedbf9e778a42e5b99d1450b15dcd847`).
- **Scores** are the program's own. They depend on the method and threshold, so repeat the run and use your numbers if they differ.
- **Times** in the log are whole-program times including about 60 ms of Java start-up.

## The first prototype, for comparison

On 22 Oct 2026 the first prototype scored T-01 25.00%, T-02 52.27%, T-06 100.00%, T-08 93.55% and T-09 62.50%, compared only two files at a time, and had no flag. The final build fixes those problems; the old results stay on the `Prototype Dry Run` sheet.